WEEK 5

AIM : To implement, train, and evaluate a Decision Tree Classifier to accurately predict categorical class labels based on feature splitting criteria and visualize the resulting decision rules.

DESCRIPTION :

A Decision Tree Classifier is a supervised machine learning algorithm widely used for predictive modeling and classification tasks. It operates by breaking down a complex dataset into smaller, more manageable subsets while simultaneously developing a tree-like model of decisions and their possible consequences. The algorithm mimics human-level decision-making, making the final model highly interpretable and easy to visualize.

Core Structure of a Decision Tree

- Root Node: Represents the entire dataset, which is then divided into two or more homogeneous sets based on the most significant feature.
- Decision Nodes (Internal Nodes): These nodes represent a specific test or condition on an independent variable (feature). Depending on the outcome of this test, the tree branches off into further sub-nodes.
- Branches: Represent the outcome of a test, connecting one node to the next.
- Leaf Nodes (Terminal Nodes): The final endpoints of the tree that do not split any further. These nodes carry the final categorical class label predicted by the model.

Splitting Criteria To construct the tree, the algorithm must determine the most useful feature to split on at every step. It evaluates the data using mathematical metrics to maximize class separation:

- Entropy: A measure of randomness or impurity within a specific subset of data. A pure node (containing only one class) has an entropy of zero.
- Information Gain: Calculated by comparing the entropy of the dataset before and after a split. The algorithm selects the feature that provides the highest Information Gain, effectively reducing the most uncertainty.
- Gini Impurity: An alternative to Entropy (commonly used by the CART algorithm) that calculates the probability of a randomly chosen instance being misclassified. The algorithm favors splits that minimize Gini Impurity.

Dataset :

EnjoySport.csv

Day	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes
5	Sunny	Warm	Normal	Strong	Warm	Same	Yes
6	Rainy	Cold	High	Strong	Warm	Change	No
7	Sunny	Warm	High	Strong	Cool	Change	Yes
8	Sunny	Warm	Normal	Weak	Warm	Same	Yes
9	Rainy	Cold	High	Strong	Cool	Change	No
10	Sunny	Warm	Normal	Strong	Warm	Same	Yes
11	Sunny	Warm	High	Weak	Warm	Same	Yes
12	Sunny	Warm	High	Strong	Warm	Change	Yes
13	Rainy	Cold	High	Weak	Warm	Change	No
14	Sunny	Warm	Normal	Weak	Cool	Change	Yes

Code :

```
import pandas as pd
import numpy as np

def calculate_entropy(target_col):
    elements, counts = np.unique(target_col, return_counts=True)
    probabilities = counts / counts.sum()
    entropy = -np.sum(probabilities * np.log2(probabilities))
    return entropy

def calculate_information_gain(data, split_attribute_name, target_name):
    total_entropy = calculate_entropy(data[target_name])
    vals, counts = np.unique(data[split_attribute_name],
return_counts=True)
    weighted_entropy = 0
    for i in range(len(vals)):
        subset = data.where(data[split_attribute_name] ==
vals[i]).dropna()
        subset_entropy = calculate_entropy(subset[target_name])
        weighted_entropy += (counts[i] / np.sum(counts)) * subset_entropy
    information_gain = total_entropy - weighted_entropy
    return information_gain

def build_decision_tree(data, original_data, features,
target_attribute_name, parent_node_class=None):
    if len(np.unique(data[target_attribute_name])) <= 1:
        return np.unique(data[target_attribute_name])[0]
    elif len(data) == 0:
        return
np.unique(original_data[target_attribute_name])[np.argmax(np.unique(origin
al_data[target_attribute_name], return_counts=True)[1])]
    elif len(features) == 0:
        return parent_node_class
    else:
        parent_node_class =
np.unique(data[target_attribute_name])[np.argmax(np.unique(data[target_att
ribute_name], return_counts=True)[1])]
        item_values = [calculate_information_gain(data, feature,
target_attribute_name) for feature in features]
        best_feature_index = np.argmax(item_values)
```

```
        best_feature = features[best_feature_index]
        tree = {best_feature: {}}
        features = [i for i in features if i != best_feature]
        for value in np.unique(data[best_feature]):
            sub_data = data.where(data[best_feature] == value).dropna()
            subtree = build_decision_tree(sub_data, original_data,
features, target_attribute_name, parent_node_class)
            tree[best_feature][value] = subtree
        return tree

def print_tree(tree_dict, indent=0):
    for key, value in tree_dict.items():
        print('    ' * indent + "|-- " + str(key))
        if isinstance(value, dict):
            print_tree(value, indent + 1)
        else:
            print('    ' * (indent + 1) + '-> Predict: ' + str(value))

file_path = "/content/Day    Sky    AirTemp    Humidity    Wind    Wate...
- Day    Sky    AirTemp    Humidity    Wind    Wate....csv"
df = pd.read_csv(file_path)
features = df.columns[1:-1].tolist()
target_col = df.columns[-1]
decision_tree = build_decision_tree(df, df, features, target_col)
print_tree(decision_tree)
```

Output :

```
|-- Sky
    |-- Rainy
        -> Predict: No
    |-- Sunny
        -> Predict: Yes
```

Salaries.csv

company	job	degree	salary_more_then_100k
google	sales executive	bachelors	0
google	sales executive	masters	0
google	business manager	bachelors	1
google	business manager	masters	1
google	computer programmer	bachelors	0
google	computer programmer	masters	1
abc pharma	sales executive	masters	0
abc pharma	computer programmer	bachelors	0
abc pharma	business manager	bachelors	0
abc pharma	business manager	masters	1
facebook	sales executive	bachelors	1
facebook	sales executive	masters	1
facebook	business manager	bachelors	1
facebook	business manager	masters	1
facebook	computer programmer	bachelors	1
facebook	computer programmer	masters	1

Code :

```
import pandas as pd
import numpy as np

def calculate_entropy(target_col):
    elements, counts = np.unique(target_col, return_counts=True)
    probabilities = counts / counts.sum()
    entropy = -np.sum(probabilities * np.log2(probabilities))
    return entropy

def calculate_information_gain(data, split_attribute_name, target_name):
    total_entropy = calculate_entropy(data[target_name])
    vals, counts = np.unique(data[split_attribute_name],
return_counts=True)
    weighted_entropy = 0
    for i in range(len(vals)):
        subset = data.where(data[split_attribute_name] ==
vals[i]).dropna()
        subset_entropy = calculate_entropy(subset[target_name])
        weighted_entropy += (counts[i] / np.sum(counts)) * subset_entropy
    information_gain = total_entropy - weighted_entropy
    return information_gain

def build_decision_tree(data, original_data, features,
target_attribute_name, parent_node_class=None):
    if len(np.unique(data[target_attribute_name])) <= 1:
        return np.unique(data[target_attribute_name])[0]
    elif len(data) == 0: return
np.unique(original_data[target_attribute_name])[np.argmax(np.unique(origin
al_data[target_attribute_name], return_counts=True)[1])]
    elif len(features) == 0:
        return parent_node_class
    else:
        parent_node_class =
np.unique(data[target_attribute_name])[np.argmax(np.unique(data[target_att
ribute_name], return_counts=True)[1])]
        item_values = [calculate_information_gain(data, feature,
target_attribute_name) for feature in features]
        best_feature_index = np.argmax(item_values)
        best_feature = features[best_feature_index]
```

```
        tree = {best_feature: {}}
        features = [i for i in features if i != best_feature]
        for value in np.unique(data[best_feature]):
            sub_data = data.where(data[best_feature] == value).dropna()
            subtree = build_decision_tree(sub_data, original_data,
features, target_attribute_name, parent_node_class)
            tree[best_feature][value] = subtree
        return tree

def print_tree(tree_dict, indent=0):
    for key, value in tree_dict.items():
        print('    ' * indent + "|-- " + str(key))
        if isinstance(value, dict):
            print_tree(value, indent + 1)
        else:
            print('    ' * (indent + 1) + '-> Predict: ' + str(value))
file_path = "/content/salaries (1).xlsx - salaries (1).csv"
df = pd.read_csv(file_path)
features = df.columns[1:-1].tolist()
target_col = df.columns[-1]
decision_tree = build_decision_tree(df, df, features, target_col)
print_tree(decision_tree)
```

Output :

```
|-- job
    |-- business manager
        |-- degree
            |-- bachelors
                -> Predict: 1.0
            |-- masters
                -> Predict: 1.0
    |-- computer programmer
        |-- degree
            |-- bachelors
                -> Predict: 1.0
            |-- masters
                -> Predict: 1.0
    |-- sales executive
        |-- degree
            |-- bachelors
                -> Predict: 0.0
            |-- masters
                -> Predict: 0.0
```

Code :

```
import pandas as pd
import numpy as np

def calculate_entropy(target_col):
    elements, counts = np.unique(target_col, return_counts=True)
    probabilities = counts / counts.sum()
    entropy = -np.sum(probabilities * np.log2(probabilities))
    return entropy

def calculate_information_gain(data, split_attribute_name, target_name):
    total_entropy = calculate_entropy(data[target_name])
    vals, counts = np.unique(data[split_attribute_name],
return_counts=True)
    weighted_entropy = 0
    for i in range(len(vals)):
        subset = data.where(data[split_attribute_name] ==
vals[i]).dropna()
        subset_entropy = calculate_entropy(subset[target_name])
        weighted_entropy += (counts[i] / np.sum(counts)) * subset_entropy
    information_gain = total_entropy - weighted_entropy
    return information_gain

def build_decision_tree(data, original_data, features,
target_attribute_name, parent_node_class=None):
    if len(np.unique(data[target_attribute_name])) <= 1:
        return np.unique(data[target_attribute_name])[0]
    elif len(data) == 0:
        return
np.unique(original_data[target_attribute_name])[np.argmax(np.unique(origin
al_data[target_attribute_name], return_counts=True)[1])]
    elif len(features) == 0:
        return parent_node_class
    else:
        parent_node_class =
np.unique(data[target_attribute_name])[np.argmax(np.unique(data[target_att
ribute_name], return_counts=True)[1])]
        item_values = [calculate_information_gain(data, feature,
target_attribute_name) for feature in features]
        best_feature_index = np.argmax(item_values)
```

```
        best_feature = features[best_feature_index]
        tree = {best_feature: {}}
        features = [i for i in features if i != best_feature]
        for value in np.unique(data[best_feature]):
            sub_data = data.where(data[best_feature] == value).dropna()
            subtree = build_decision_tree(sub_data, original_data,
features, target_attribute_name, parent_node_class)
            tree[best_feature][value] = subtree
        return tree

def print_tree(tree_dict, indent=0):
    for key, value in tree_dict.items():
        print('    ' * indent + "|-- " + str(key))
        if isinstance(value, dict):
            print_tree(value, indent + 1)
        else:
            print('    ' * (indent + 1) + '-> Predict: ' + str(value))
file_path = "/content/titanic (1).xlsx - titanic (1).csv"
df = pd.read_csv(file_path)
df = df.dropna()
columns_to_drop = ['PassengerId', 'Name', 'Ticket', 'Cabin']
df = df.drop(columns=[col for col in columns_to_drop if col in
df.columns])
if 'Survived' in df.columns:
    target_col = 'Survived'
    features = df.columns.tolist()
    features.remove('Survived')
else:
    features = df.columns[:-1].tolist()
    target_col = df.columns[-1]
decision_tree = build_decision_tree(df, df, features, target_col)
print("Generated Decision Tree Rules:\n")
print_tree(decision_tree)
```

Output :

```
Generated Decision Tree Rules:

|-- Fare
    |-- 0.0
        -> Predict: 0.0
    |-- 5.0
        -> Predict: 0.0
```

```
|-- 7.65
    -> Predict: 0.0
|-- 8.05
    -> Predict: 1.0
|-- 10.4625
    -> Predict: 0.0
|-- 10.5
    |-- Age
        |-- 27.0
            -> Predict: 1.0
        |-- 29.0
            -> Predict: 1.0
        |-- 34.0
            -> Predict: 1.0
        |-- 57.0
            -> Predict: 0.0
|-- 12.475
    -> Predict: 1.0
|-- 12.875
    -> Predict: 0.0
|-- 13.0
    -> Predict: 1.0
|-- 13.7917
    -> Predict: 1.0
|-- 16.7
    -> Predict: 1.0
|-- 25.5875
    -> Predict: 0.0
|-- 25.9292
    -> Predict: 1.0
|-- 26.0
    |-- Age
        |-- 2.0
            -> Predict: 1.0
        |-- 3.0
            -> Predict: 1.0
        |-- 36.5
            -> Predict: 0.0
|-- 26.2833
    -> Predict: 1.0
|-- 26.2875
    -> Predict: 1.0
|-- 26.3875
    -> Predict: 1.0
|-- 26.55
    |-- Age
        |-- 28.0
```